Mini-Project 1

University: Mansoura , Engineering School

Department: Electronic and Communication Engineering

Course: Electronics Summer Internship

Student Name: Menna Harby Elsherbeiny Ramadan Elsherbeiny

منةالله حربي الشربيني رمضان الشربيني

Student ID: 1000296289

Deadline: July 26 , 2025

Notion Page: Mini-Project 1

GitHub: https://github.com/Menna-Harby/Electronics-Internship.git

Smart Room Lighting System

📌 Objective

To design and implement an Arduino-based **smart lighting system** that:

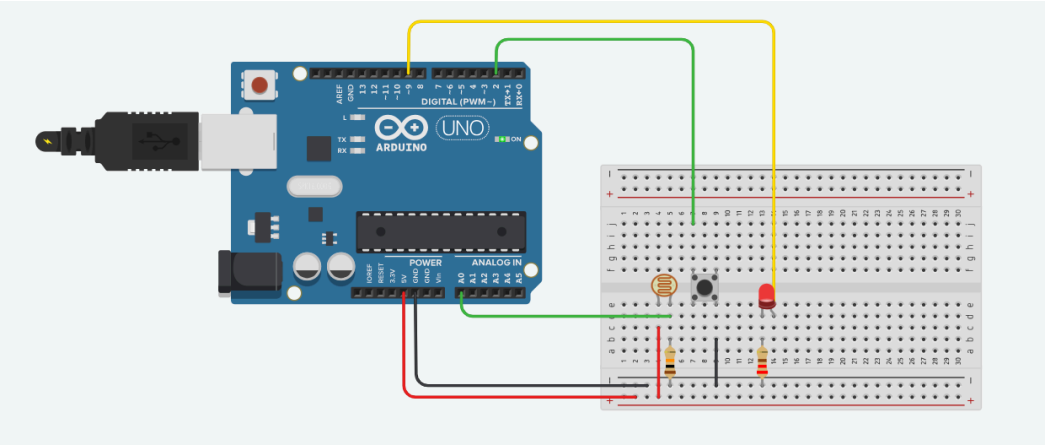
- Automatically adjusts LED brightness based on ambient light.
- Allows manual control using a button.
- Implements debouncing for button reliability.
- Saves energy with an auto-timeout feature when no activity is detected.

🛠️ Components Used

Component	Quantity	Description	Cost	store
Arduino Uno	1	Microcontroller	400 EGP	https://mecha-tronx.com/product/arduino-uno-r3/
LED	1	Output light source	0.50 EGP	https://mecha-tronx.com/product/led-5mm/
Photoresistor (LDR)	1	Detects ambient light	15 EGP	https://mecha-tronx.com/product/photo-resistor-sensor-ldr-7mm/
10kΩ Resistor	1	For LDR voltage divider	3 EGP	
Push Button	1	User input	1 EGP	
220Ω Resistor	1	Current limit for LED	3 EGP	
Breadboard + Wires	-	For prototyping	12 EGP	https://mecha-tronx.com/product/170pts-mini-breadboard-syb-170/

> **Total Cost: 440 EGP**

🔌 Circuit Diagram



🚧 **Wiring Summary:**

1. **LDR Circuit**:
   - Connect one leg of LDR to 5V
   - Connect other leg to analog pin A0 and to GND through 10kΩ resistor
2. **LED Circuit**:
   - Connect LED anode to PWM pin 9 through 220Ω resistor
   - Connect LED cathode to GND
3. **Button Circuit**:
   - Connect one leg of button to digital pin 2
   - Connect other leg to GND

💡 System Behavior Description

Mode	Trigger	Behavior
Auto Mode	Default mode	LED brightness automatically adjusts based on light.
Manual Mode	Press button	Cycles brightness: 25% → 50% → 75% → 100% → OFF.
Debouncing	Built-in in software	Prevents false readings from button bouncing.
Timeout	2 minutes of inactivity	Turns off LED to save energy.
Reactivation	Press button after timeout	Resumes last mode and restores function.

👨‍💻 Code (with Comments & Explanation)

```
#define ldrPin A0
#define ledPin 9
```

```
#define buttonPin 2

bool autoMode = true;
int manualLevel = 0;
const int levels[] = {0, 64, 128, 192, 255}; // 0%, 25%, 50%, 75%, 100%

unsigned long lastDebounceTime = 0;
unsigned long debounceDelay = 50;
bool lastButtonState = HIGH;
bool buttonState = HIGH;

unsigned long lastActivityTime = 0;
unsigned long timeoutDuration = 120000; // 2 minutes
bool isSystemActive = true;
int lastLDRValue = 0;
const int ldrChangeThreshold = 20;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP); // Active LOW
  Serial.begin(9600);
  lastActivityTime = millis();
  lastLDRValue = analogRead(ldrPin);
}

void loop() {

// ---------------Button Press Detection with Debouncing------------------
  int reading = digitalRead(buttonPin);

  // Debounce logic
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }

  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading != buttonState) {
      buttonState = reading;

      if (buttonState == LOW) {
        // Button press detected
        lastActivityTime = millis();
        isSystemActive = true;

        if (autoMode) {
          autoMode = false; // Switch to manual mode
          manualLevel = 1;  // Start at 25%
        } else {
          // Cycle through manual brightness levels
          manualLevel = (manualLevel + 1) % 5;
          if (manualLevel == 0) {
            autoMode = true; // If OFF (0%), return to auto mode
```

```
        }
      }
    }
  }
}

lastButtonState = reading;

// ------------------Ambient Light Change Detection--------------------

// Detect LDR changes only if system is active
if (isSystemActive && autoMode) {
  int ldrValue = analogRead(ldrPin);
  if (abs(ldrValue - lastLDRValue) > ldrChangeThreshold) {
    lastActivityTime = millis(); // Significant ambient change
    lastLDRValue = ldrValue;
  }
}

// -------------------------Inactivity Timeout Check---------------
if (isSystemActive && (millis() - lastActivityTime > timeoutDuration)) {
  analogWrite(ledPin, 0); // Turn off LED
  isSystemActive = false;
  Serial.println("System timed out. LED off.");
}

// --------------------------Brightness Control-----------------------

if (isSystemActive) {
  if (autoMode) {
    int ldrValue = analogRead(ldrPin); // 0 (bright) to 1023 (dark)
    int brightness = map(ldrValue, 1023, 0, 0, 255); // Invert mapping
    analogWrite(ledPin, brightness);
    Serial.print("Auto Brightness: ");
    Serial.println(brightness);
  } else {
    analogWrite(ledPin, levels[manualLevel]);
    Serial.print("Manual Brightness: ");
    Serial.println(levels[manualLevel]);
  }
}

delay(50);
}
```

Pin Definitions and Global Variables

```
#define ldrPin A0
                  // Analog input pin connected to LDR
#define ledPin 9
                 // PWM-capable digital pin connected to LED
```

```
#define buttonPin 2
                // Digital pin connected to push button (with internal pull-up)



bool autoMode = true;
          // Tracks whether system is in automatic (true) or manual (false) mode
int manualLevel = 0;
          // Current brightness level index (0-4) in manual mode
const int levels[] = {0, 64, 128, 192, 255}; // 0%, 25%, 50%, 75%, 100%
         // Array of PWM values representing different brightness levels (0-255)


// ----------- Debounce variables ------------------
unsigned long lastDebounceTime = 0;
           // When the last button state change was detected
unsigned long debounceDelay = 50;
           // Minimum time (ms) between button state changes
bool lastButtonState = HIGH;
          // Previous reading from the button
bool buttonState = HIGH;
         // Current debounced button state


 //-------------- Timeout variables ----------------------
unsigned long lastActivityTime = 0;
unsigned long timeoutDuration = 120000; // 2 minutes
bool isSystemActive = true;
// ==⇒ LDR variables
int lastLDRValue = 0;
const int ldrChangeThreshold = 20; // Minimum change in LDR value to count as activity
```

Setup Function

```
void setup() {
  pinMode(ledPin, OUTPUT);    // Sets LED pin as output
  pinMode(buttonPin, INPUT_PULLUP);
        // Configures button pin with internal pull-up resistor (button press = LOW)
  Serial.begin(9600);
  lastActivityTime = millis();   // Initializes last activity time to current time
  lastLDRValue = analogRead(ldrPin);
}
```

Main Loop

🔍 Debounce Logic Explanation

Purpose: Prevent multiple detections from a single physical button press caused by bouncing.

Steps:

1. Read the button value.

2. If it **changed** from last time → mark current time.
3. If the **new value stays stable for 50 ms**, accept it as real.
4. If the button is pressed (LOW), act on it.

This ensures:

- One press = one action.
- No flickering or multiple triggers.

```
if (autoMode) {
        autoMode = false; // Switch to manual mode
        manualLevel = 1;  // Start at 25%
      } else {
        // Cycle through manual brightness levels
        manualLevel = (manualLevel + 1) % 5;
        if (manualLevel == 0) {
          autoMode = true; // If OFF (0%), return to auto mode
        }
      }
```

1. `manualLevel + 1` moves to the next level
   - If current is 1 (25%), next is 2 (50%)
2. `% 5` is the "modulo" operation - it makes the number wrap around after 4
   - Think of it like a clock that goes 0→1→2→3→4→0→1...

Example Walkthrough:

Let's say we're in manual mode at 25% brightness (level 1):

1. First button press:
   - `manualLevel = (1 + 1) % 5` → `2 % 5` → 2 (50%)
2. Second button press:
   - `manualLevel = (2 + 1) % 5` → `3 % 5` → 3 (75%)
3. Third button press:
   - `manualLevel = (3 + 1) % 5` → `4 % 5` → 4 (100%)
4. Fourth button press:
   - `manualLevel = (4 + 1) % 5` → `5 % 5` → 0 (OFF)
5. Fifth button press:
   - Now the `if (manualLevel == 0)` condition is true, so it switches back to auto mode

---

🎬 System Demonstration

https://www.tinkercad.com/things/cFER6QMDuKE/editel?returnTo=%2Fdashboard%2Fdesigns%2F3d&sharecode=e1ny9mSIXY2BdEtQvueQUJlqsHscp02OgsDnBYy-0sk

---

✅ Conclusion

This project successfully implements a **smart lighting system** that is:

- Energy-efficient (with timeout),
- Reliable (debounced input),
- Adaptive (ambient light response),
- User-friendly (manual override).

It demonstrates practical use of:

- **Analog sensors (LDR)**
- **PWM control**
- **Debounce software**
- **Timer-based state handling**

📄References

[1] Arduino Documentation, "millis() – Time," *Arduino*, [Online]. Available: https://docs.arduino.cc/language-reference/en/functions/time/millis/. [Accessed: Jul. 24, 2025].

[2] Arduino Documentation, "Debounce – Built-in Examples," *Arduino*, [Online]. Available: https://docs.arduino.cc/built-in-examples/digital/Debounce/. [Accessed: Jul. 24, 2025].

[3] Pico Technology, "What is switch bounce and how to implement debounce," *Pico Technology Blog*, [Online]. Available: https://www.picotech.com/library/articles/blog/what-is-switch-bounce-how-to-implement-debounce. [Accessed: Jul. 24, 2025].